

Chapter 3: Server-Side Web Programming

By: Er. Ukesh Thapa

Syllabus

3.1 MVC architecture in web development

3.2 Role of backend in web applications

3.3 Backend web framework: Django

3.4 Handling requests and responses

3.5 Form data handling and sessions

3.6 Routing, middleware, templating concepts

3.7 Overview and comparison of backend frameworks: Django, flask, fastapi, dot NET MVC framework, ruby on rails, java spring boot, node.js

3.8 Database integration: Relational versus NoSQL, CRUD operations, ORM concept

3.9 Authentication and authorization: Cookies, sessions, JWT

3.10 Middleware for logging, error handling, and security

Server Side Programming

- ❖ Server-side programming is the code that runs on the web server, not on the user's computer (the client).
- ❖ It is the "brain" behind the application
- ❖ Importance:
 - **Authentication & Security:** Checking if a password is correct.
 - **Database Interaction:** Reading/Writing data (e.g., Saving a student's grade).
 - **Dynamic Content:** assembling HTML on the fly. For example, when you log into Facebook, the server builds a page specifically for you with your friends' posts.
 - **API Management:** Sending JSON data to mobile apps or React frontends.

Server Side Methods

HTTP Method	CRUD Operation	Description	Example URL
GET	Read	Ask the server to <i>send</i> data. Safe to repeat.	GET /students/101
POST	Create	Send data to the server to <i>create</i> a new record.	POST /students/
PUT	Update	Replace an <i>entire</i> record with new data.	PUT /students/101
PATCH	Update	Update <i>part</i> of a record (e.g., just change the email).	PATCH /students/101
DELETE	Delete	Remove a record.	DELETE /students/101

Server Side Programming: Technologies

Python	:	Django, FastAPI
JavaScript	:	Node.js (Express)
Java	:	Spring Boot
PHP	:	Laravel
C#	:	ASP.NET Core

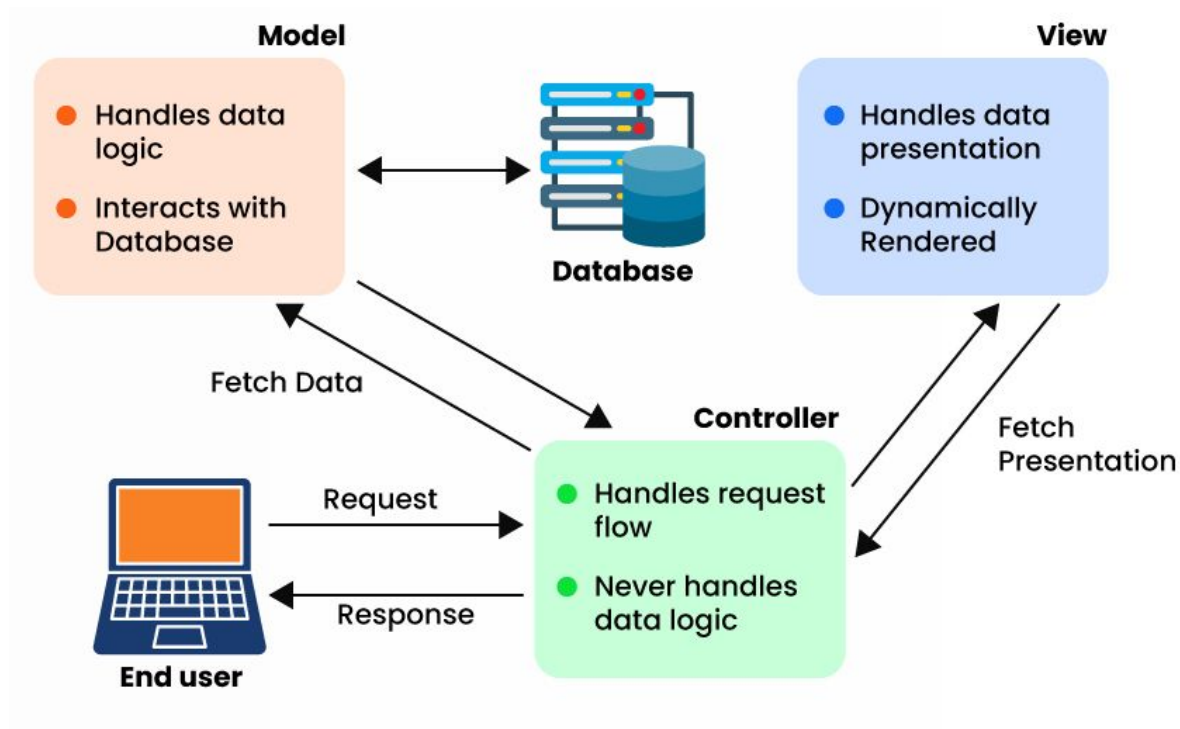
MVC Architecture in Web Development

- ❖ The MVC pattern was invented by a **Norwegian** computer scientist named **Trygve Reenskaug** in **1979**.
- ❖ A way for users to manipulate complex data sets (like a spreadsheet or a drawing) without the code becoming a mess.
- ❖ Separating the system into three parts:
 - Model: The data.
 - View: The visual representation.
 - Controller: The link between the user and the system.
- ❖ Originally, he called it Thing-Model-View-Editor
- ❖ It is a software design pattern that separates application into interconnected parts: Model, View, and Controller

Features of MVC

- ❖ It provides a clear separation of business logic, UI logic, and input logic.
- ❖ It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- ❖ It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- ❖ It supports Test Driven Development (TDD).

MVC Architecture



Model

- ❖ The Model component corresponds to all the data-related logic that the user works with.
- ❖ This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.
- ❖ It can add or retrieve data from the database.
- ❖ Responsibilities:
 - Managing data: CRUD (Create, Read, Update, Delete) operations.
 - Enforcing business rules.
 - Notifying the View and Controller of state changes.

View

- ❖ The View component is used for all the UI logic of the application.
- ❖ It generates a user interface for the user.
- ❖ Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller.
- ❖ Responsibilities:
 - Rendering data to the user in a specific format.
 - Displaying the user interface elements.
 - Updating the display when the Model changes.

Controller

- ❖ The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary.
- ❖ The controller doesn't have to worry about handling data logic, it just tells the model what to do.
- ❖ It processes all the business logic and incoming requests, manipulates data using the Model component, and interact with the View to render the final output.
- ❖ Responsibilities:
 - Receiving user input and interpreting it.
 - Updating the Model based on user actions.
 - Selecting and displaying the appropriate View.

Web MVC

- ❖ Java Struts (2000): One of the first major frameworks to force MVC on the web.
- ❖ Spring (2002) & Ruby on Rails (2004): These frameworks popularized the modern "Request-Response" MVC we use today.
- ❖ Django (2005): Adopted the pattern but renamed it MVT (Model-View-Template), though the concept remains identical.

Backend Development

- ❖ Backend development handles the server-side logic, database, and core functionality of web applications.
- ❖ It ensures smooth communication between the server and client using languages and frameworks like Node.js, Django, and Spring.
- ❖ **Importance:**
 - Security
 - Centralization
 - Heavy Processing

Role of Backend in Web Applications

❖ Server Logic

This is the technical layer that handles the HTTP Protocol. It deals with the raw mechanism of the web.

Responsibilities

- Request Parsing: Converting the raw data stream (bytes) from the browser into a usable programming object (like a Python Dictionary).
- Response Formatting: Packaging the result from the Business Logic into a format the browser understands (HTML, JSON, XML).
- Status Codes: Assigning the correct HTTP signal (e.g., 200 OK, 404 Not Found, 500 Server Error).

Role of Backend in Web Applications

❖ Routing

Routing is the mechanism that maps the specific URL (address) a user visits to the specific function or class in your code that handles that request.

Responsibilities

- URL Matching: Comparing the incoming URL string against a list of known patterns.
- Parameter Extraction: Pulling dynamic data out of the URL. If the URL is /students/105/, the router extracts 105 as the ID.
- Traffic Direction: Calling the correct "View" or "Controller" function.

Role of Backend in Web Applications

❖ Security

The Security layer is a series of checks and validations designed to protect the application from unauthorized access and malicious attacks before any real work begins.

Responsibilities

- ❖ **Authentication (Who are you?):** Verifying identity (e.g., checking session cookies or JWT tokens).
- ❖ **Authorization (Are you allowed?):** Checking permissions (e.g., ensuring a "Student" cannot access "Teacher" pages).
- ❖ **Sanitization:** Cleaning input data to prevent hacking attempts like SQL Injection or XSS (Cross-Site Scripting).

Role of Backend in Web Applications

Three Layers of Security:

1. **Encryption** : Way of securing data along with a password or “key” which is used to decrypt the information.
2. **Data Integrity**: When data integrity is secure, information stored in a database is complete, accurate, and reliable for a lifetime.
3. **Authentication** : This method protects against attacks and builds user trust which response to business growth.

Role of Backend in Web Applications

❖ Business Logic

Business Logic represents the real-world rules, calculations, and workflows specific to your organization's domain. This is the core value of your software; it is what distinguishes a "School App" from a "Banking App."

Responsibilities

- ❖ **Rule Enforcement:** Checking conditions (e.g., "Cannot withdraw more than current balance").
- ❖ **Data Processing:** Performing calculations, generating reports, or modifying data.
- ❖ **Database Interaction:** Deciding exactly what to save, update, or delete in the database based on the rules.

Django Framework

- ❖ Django is a Python framework that makes it easier to create web sites using Python.
- ❖ Django takes care of the difficult stuff so that you can concentrate on building your web applications.
- ❖ Django emphasizes reusability of components, also referred to as DRY (Don't Repeat Yourself)
- ❖ Django has Ready-to-use features like login system, database connection and CRUD operations (Create Read Update Delete).

Django Framework: MVT

- ❖ Stands for Model View Template
- ❖ "Controller" to decide which view loads for which URL. So, all these tasks are done by the framework itself, you don't need to write a controller.
- ❖ The Components:
 - Model:
 - The Data Manager
 - The Model is the single source of truth about your data. It contains the essential fields and behaviors of the data you're storing.
 - Role: Handles all database interactions (Create, Read, Update, Delete).
 - File: models.py
 - Key Concept (ORM): Django uses an Object-Relational Mapper. Students write Python classes, and Django translates them into SQL commands automatically.

Django Framework: MVT

❖ View:

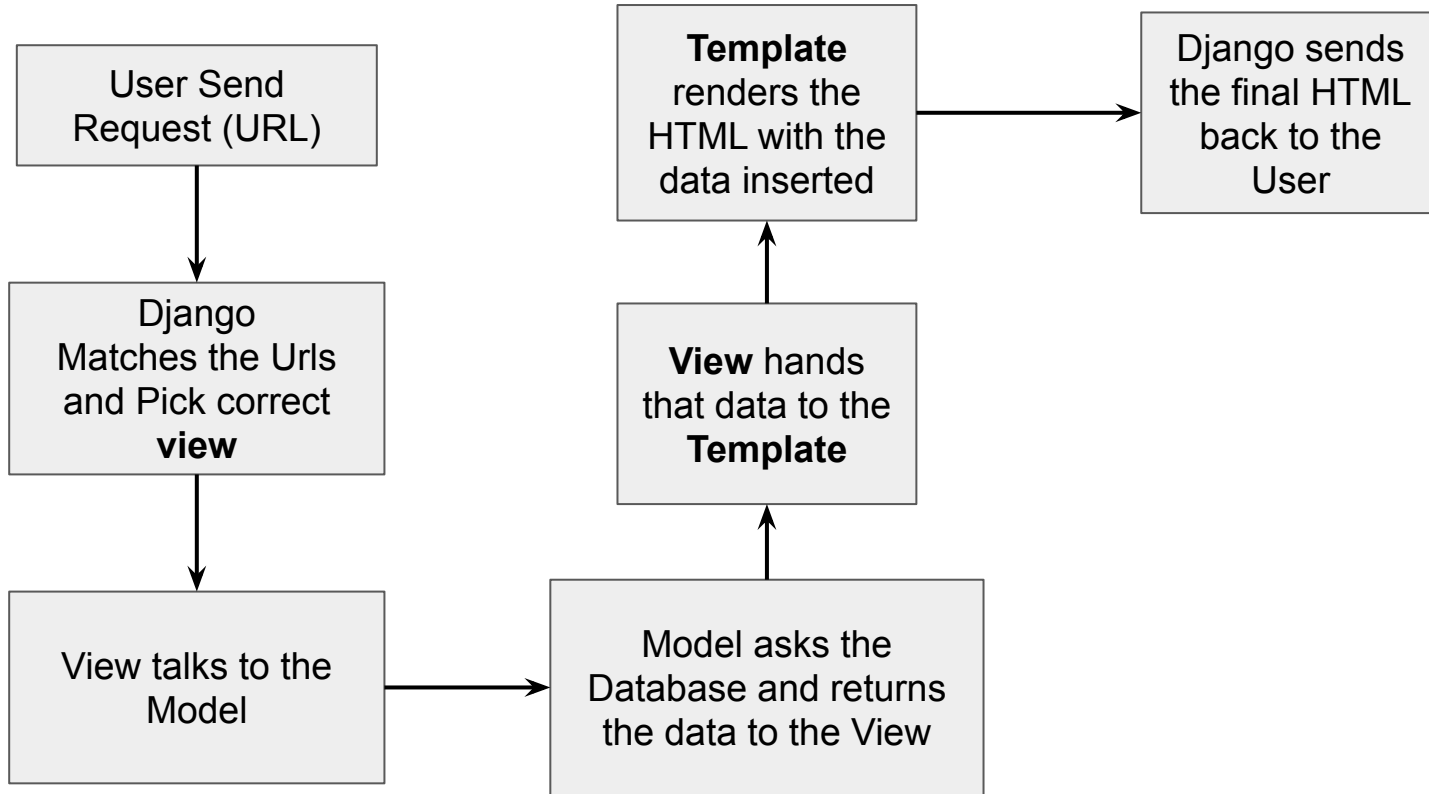
- The logic layers
- The View is a Python function or class that accepts a web request and returns a web response.
- It acts as the bridge between the Model and the Template.
 - Role: Fetches data from the Model, processes it (filters, calculations), and sends it to the Template.
 - File: views.py
 - **Responsibilities:**
 - ◆ Receive request object.
 - ◆ Check permissions (Is the user logged in?).
 - ◆ Query the Database (using the Model).
 - ◆ Return a HttpResponse (usually by rendering a Template).

Django Framework: MVT

❖ Template:

- The Template is a text file defining the structure or layout of the file (usually HTML).
- It knows how to display data, but not how to get it.
 - Role: Displays the data passed to it by the View. It contains placeholders for dynamic content.
 - File: **.html** files in the templates folder.
 - Key Concept (DTL): Django Template Language. It allows logic inside HTML (loops, if-statements) but is deliberately limited to prevent complex logic from leaking into the presentation.

Django Framework: WorkFlow



Django Framework

Feature	Django Project	Django App
Definition	The entire web application or website. It is the container for everything.	A submodule of the project that does one specific thing .
Scope	Global. It handles the whole configuration.	Specific. It handles a distinct feature (e.g., Blog, Forum, Payments).
Quantity	There is only 1 Project per website.	There can be Many Apps inside one Project.
Reusability	Not Reusable . It is unique to your specific website deployment.	Highly Reusable . You can plug a "Blog App" into 5 different projects without changing code.
Key Files	<code>settings.py</code> , <code>urls.py</code> (main routing), <code>wsgi.py</code> (deployment).	<code>models.py</code> , <code>views.py</code> , <code>admin.py</code> , <code>tests.py</code> .
Command	<code>django-admin startproject mywebsite</code>	<code>python manage.py startapp blog</code>
Analogy	The Smartphone.	The Applications installed on the phone (Camera, Maps, Contacts).

Django Project Structure

- ❖ `manage.py` → project controller
- ❖ `settings.py` → configuration
- ❖ `urls.py` → request routing
- ❖ `views.py` → logic

```
project/  
├── manage.py  
├── project/  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └── wsgi.py  
├── app/  
│   ├── views.py  
│   ├── models.py  
│   ├── urls.py  
│   └── templates/
```

Django Framework: Startup

1. Create virtualenv

```
D:\Course\Web Application Programming\assessments\Chapter 3\Django>virtualenv env
```

2. Install Django

```
pip install django python-dotenv
```

3. Project Creation

```
django-admin startproject config .
```

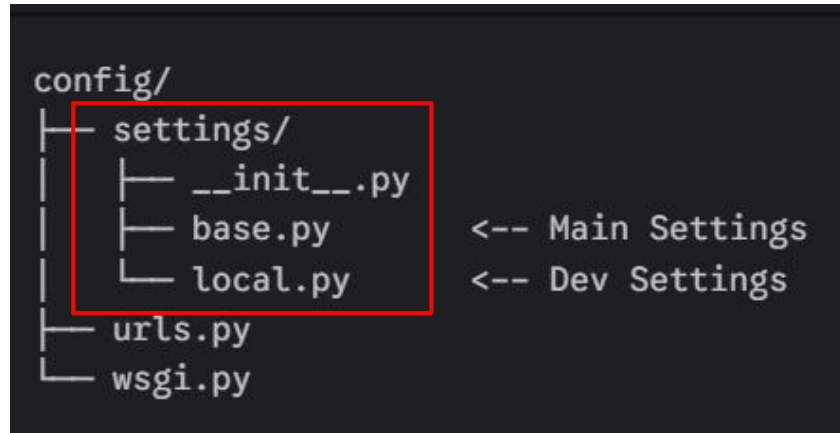
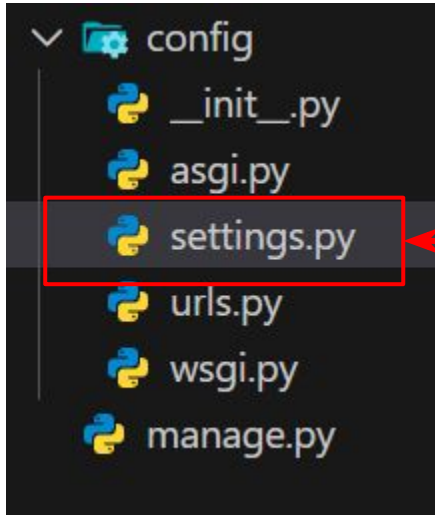
Django Framework: Startup

4. Create the Structure

I. Delete the existing config/settings.py.

II. Create a folder config/settings/.

III. Create two files inside it: `__init__.py` (empty) and `base.py`, `local.py`.



Django Framework: Startup

5. Change in manage.py

Old → wrong

```
def main():  
    """Run administrative tasks."""  
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')  
    try:
```

Changes → Correct

```
def main():  
    """Run administrative tasks."""  
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.local')  
    try:  
        from django.core.management import execute_from_command_line  
    except ImportError as exc:
```

Django Framework: Startup

6. python manage.py startapp App

>> python manage.py startapp App

7. Register the App in (./config/settings/base.py)

```
# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Custom Apps go here (We will add one later)
    'App', # custom add
]
```

Django Framework: Startup

8. Create **url.py** in App



9. Project URLs Edit config/urls.py.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('App.urls'))
]
```

```
from django.urls import path
from . import views

app_name = 'App' # Namespacing

urlpatterns = [
    path('', views.home_view, name='home'),

    # NEW PATH
    path('hello/', views.hello_world, name='hello'),
]
```

Django Framework: Startup

10. Python Migrate

```
>> python manage.py migrate
```

(Note: To update the Django table)

11. Run Server

```
>> python manage.py runserver
```

12. Create superuser password

```
>> python manage.py createsuperuser
```

Django Model

- ❖ A Django model is a Python class that represents a database table.
- ❖ It lets developers manage data easily using Python instead of writing SQL.
- ❖ Create Table (Model)
 - To create a model, navigate to the [models.py](#)
 - Open it, and add a Member table by creating a Member class
- ❖ when we have described a Model in the **models.py** file, we must run a command to actually create the table in the database.
 - **python manage.py makemigrations**
 - **python manage.py migrate**
- ❖ Registering Models in Django Admin
 - Open admin.py in app directory.
 - Register the model: **admin.site.register(Model_Name)**

Model Field Types

- ❖ Django models consist of fields that define the structure of database tables.

Each field specifies:

- The type of data it stores (text, numbers, dates, files, etc.)
- Constraints and validations on the data

- ❖ Common field types include:

- CharField: Stores short text (like names or titles)
- TextField: Stores large amounts of text (like descriptions)
- IntegerField: Stores integer values
- BooleanField: Stores True/False values
- DateTimeField: Stores date and time
- ImageField / FileField: Stores images or files

Django ORM - Inserting, Updating & Deleting Data

- ❖ Django's ORM (Object Relational Mapper) makes database work easier by letting developers use Python classes instead of writing SQL.
 - Defines the database structure using Python models.
 - Connects Python objects with database tables.
 - Allows easy creation, reading, updating, and deletion of data.
 - Removes the need to write raw SQL for most operations
- ❖ Django ORM Queries:
 - **.all()**: Retrieve all objects of a model.
 - **.filter()**: Retrieve objects that match specific conditions.
 - **.exclude()**: Retrieve objects that do not match specific conditions.
 - **.get()**: Retrieve a single object that matches the condition (raises an error if multiple objects match).
 - **.update()**: Update multiple objects at once.
 - **.delete()**: Delete records from the database

Example:

Create Model:

1. Student Profile

```
# Create your models here.  
from django.db import models  
from django.core.validators import MinValueValidator, MaxValueValidator  
  
class StudentProfile(models.Model):  
    name = models.CharField(max_length=100)  
    roll = models.CharField(max_length=20, unique=True)  
    program = models.CharField(max_length=50)  
  
    def __str__(self):  
        return self.name
```

Example

2. StudentGrade

```
class StudentGrade(models.Model):
    student = models.ForeignKey(StudentProfile, on_delete=models.CASCADE, related_name='grades')

    # Choices for Semester
    SEM_CHOICES = [
        ('1st', '1st Semester'),
        ('2nd', '2nd Semester'),
        ('3rd', '3rd Semester'),
        ('4th', '4th Semester'),
        ('5th', '5th Semester'),
        ('6th', '6th Semester'),
        ('7th', '7th Semester'),
        ('8th', '8th Semester'),
    ]

    year = models.CharField(max_length=4)
    sem = models.CharField(max_length=10, choices=SEM_CHOICES)

    # GPA SCALE: 0.00 to 4.00
    gpa = models.DecimalField(
        max_digits=3,
        decimal_places=2,
        validators=[MinValueValidator(0.0), MaxValueValidator(4.0)],
        help_text="Enter GPA between 0.0 and 4.0"
    )

    status = models.CharField(max_length=10, editable=False) # Editable=False hides it in forms

    def save(self, *args, **kwargs):
        # Professional logic: GPA 2.0 is usually the pass mark
        if self.gpa >= 2.0:
            self.status = "Pass"
        else:
            self.status = "Fail"
        super().save(*args, **kwargs)

    def __str__(self):
        return f"{self.student.name} | {self.sem} | GPA: {self.gpa}"
```

Make Migration

python manage.py makemigrations

python manage.py migrate

First time: Create Superuser: **python manage.py createsuperuser**

Register: Add both to admin.py so you can add data via the GUI.

```
# admin.py
from django.contrib import admin
from .models import StudentProfile, StudentGrade

admin.site.register(StudentProfile)
admin.site.register(StudentGrade)
```

Writing View

- ❖ Two main type of views
 - **Function-Based Views (FBVs)**
 - Written as Python functions.
 - They are simple and ideal for small features or straightforward logic, such as displaying a list of items, handling form submissions, or showing a single detail page.
 - **Applications and Features:**
 - Displaying a list of items
 - Showing details of a single object
 - Handling basic form submissions

```
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("Hello, world!")
```

Writing View

❖ Two main type of views

➤ **Class-Based Views (CBV)**

- Written as Python classes.
- They offer better organization, reusability, and support for object-oriented features. They are suited for complex applications, reusable components, or features that require handling multiple HTTP methods cleanly.
- When mapping a CBV to a URL, you use the **.as_view()** method
- **Applications and Features:**
 - Generic CRUD operations
 - Multi-step forms
 - Features requiring multiple HTTP methods in a clean, organized structure

```
from django.views import View
from django.http import HttpResponse

class HelloWorldView(View):
    def get(self, request):
        return HttpResponse("Hello, world!")
```

Http Methods

❖ GET: The "Request for Information"

- **Purpose:** To retrieve data from the server without changing anything.
- **Data Visibility:** Data is sent in the URL (e.g., search/?q=django).
- **Django access:** `request.GET`

❖ POST: The "Submit for Action"

- **Purpose:** To send data to the server to create or update a resource.
- **Data Visibility:** Data is hidden in the "body" of the request (secure).
- **Django Access:** `request.POST`

```
from django.views.decorators.http import require_http_methods

@require_http_methods(["GET", "POST"])
def enroll_student(request):
    # 1. Logic for showing the form
    if request.method == "GET":
        return render(request, 'enroll.html')

    # 2. Logic for processing the form
    if request.method == "POST":
        name = request.POST.get('name')
        # ... logic to save to DB ...
        return redirect('success_page')
```

Json Response

- ❖ JsonResponse is a specialized version of the standard HttpResponse that tells the browser, "I am sending you raw data, not a visual webpage."
- ❖ How it work:
 - Serialization: It converts a Python dictionary or list into a JSON-formatted string.
 - Content-Type Header: It sets the HTTP header to application/json, which informs the receiving app (like a mobile app or a React frontend) how to handle the data.
- ❖ Syntax: `JsonResponse({Key : Value}, safe = True, status = 401)`
- ❖ Key parameters: **safe** (default value True), **status** (like status = 401)
- ❖ Why is it used in Backend Development?
 - **Asynchronous Updates (AJAX)**
 - **Mobile Apps:** Android and iOS apps don't understand HTML templates; they need raw JSON data to populate their native screens.
 - **Decoupling:** It allows the backend (Django) to be completely separate from the frontend (React/Vue/Next.js).

Basic Json

```
from django.http import JsonResponse
from .models import Student

def student_api_view(request, pk):
    try:
        student = Student.objects.get(pk=pk)
        data = {
            "id": student.id,
            "full_name": student.first_name + " " + student.last_name,
            "email": student.email,
            "status": "Active"
        }
        return JsonResponse(data)
    except Student.DoesNotExist:
        # Returning a JSON error message with a 404 status code
        return JsonResponse({"error": "Student not found"}, status=404)
```

List

```
def all_students_api(request):  
    # Fetching values as a list of dictionaries  
    students = list(Student.objects.values('id', 'first_name'))  
    return JsonResponse(students, safe=False)
```

Example:

Create View

```
@login_required # 1. AUTHENTICATION: Only logged-in users get data
def student_dashboard(request):
    # Fetch Data
    students = StudentProfile.objects.prefetch_related('grades').all()

    # 2. JSON API LOGIC: Check if requester wants JSON (API call)
    if request.headers.get('Accept') == 'application/json' or request.GET.get('format') == 'json':
        data = []
        for s in students:
            data.append({
                "roll": s.roll,
                "name": s.name,
                "program": s.program,
                "grades": list(s.grades.values('sem', 'gpa', 'status'))
            })
        # 3. STATUS CODE: Return 200 OK with JSON data
        return JsonResponse({"students": data}, status=200)

    # 4. HTML RESPONSE: Standard browser view
    context = {
        'students': students,
        'student_count': students.count(),
    }
    return render(request, 'dashboard.html', context, status=200)
```

Example:

Create View

```
# add students
@login_required
def add_student(request):
    if request.method == "POST":
        # 1. DATA EXTRACTION: Handle both Form Data and JSON
        if request.content_type == 'application/json':
            try:
                data = json.loads(request.body)
            except json.JSONDecodeError:
                return JsonResponse({"error": "Invalid JSON"}, status=400)
        else:
            data = request.POST

        name = data.get('name')
        roll = data.get('roll')
        program = data.get('program')

        # 2. VALIDATION
        if not name or not roll:
            if request.headers.get('Accept') == 'application/json':
                return JsonResponse({"error": "Name and Roll are required"}, status=400)
            messages.error(request, "Name and Roll Number are required!")
            return render(request, 'add_student.html')

        # 3. DATABASE OPERATION
        student = StudentProfile.objects.create(
            name=name,
            roll=roll,
            program=program
        )

        # 4. RESPONSE LOGIC (React vs HTML)
        if request.headers.get('Accept') == 'application/json':
            return JsonResponse({
                "message": "Student created successfully",
                "id": student.id
            }, status=201) # 201 Created is the standard for new resources

        messages.success(request, f"Student {name} added!")
        return redirect('studentdashboard:student_dashboard')

# GET Request: Just show the HTML form
return render(request, 'add_student.html')
```

Routing (URL Dispatcher)

- ❖ Django looks at the URL path from top to bottom. As soon as it finds a match, it stops and sends the request to the associated View.
- ❖ The Three Parts of a Route
 - **The Route ('student/add/'):** The actual text the user types in the browser address bar.
 - **The View (views.add_student):** The Python function that contains the logic (the "Kitchen" that cooks the data).
 - **The Name ('add_student'):** A nickname. This is powerful because if you change the URL later to /register-new/, you don't have to update your HTML links; you just keep using the nickname.

```
path('student/add/', views.add_student, name='add_student')
```

Example:

Create urls

```
urlpatterns = [  
    path('', views.home_page, name = "home"),  
    # The dashboard URL (referenced in your view's redirect)  
    path('login/', auth_views.LoginView.as_view(template_name='login.html'), name='login'),  
    path('dashboard/', views.student_dashboard, name='student_dashboard'),  
    path('student/add/', views.add_student, name='add_student'),  
]
```

Settings Update

```
# Authentication Settings  
LOGIN_URL = 'studentdashboard:login' # The 'name' of your login path in urls.py  
LOGIN_REDIRECT_URL = 'studentdashboard:student_dashboard' # Redirects here after successful login  
LOGOUT_REDIRECT_URL = 'studentdashboard:login' # Redirects here after clicking logout
```

Template Language

❖ Variables `{{ ... }}` (The "Show Me" Tool)

- Use double curly braces when you want to print a piece of data directly onto the screen.
- **Example:** Welcome, `{{ user.username }}`!
- Result: Welcome, Student101!

❖ Tags `{% ... %}`

- Use curly braces with percent signs for "programming logic" like loops, if-statements, or linking pages.
 - **For Loops:** To list every student in a table. `{% for x in list%}...{% endfor%}`
 - **If Statements:** To show a "Pass" or "Fail" message based on a grade. `{% if x > 5 %}...{% endif %}`
 - **URL Linking:** `{% url 'add_student' %}` finds the correct link for you.

❖ Filters `{{ value|filter }}`

- Filters allow you to modify how data looks without changing the actual data in the database.
- Example: `{{ student.name|upper }}` → JOHN DOE
- Example: `{{ date|date:"M Y" }}` → Jan 2026

Template Language

❖ Template Inheritance

- Instead of copy-pasting your Navigation Bar onto 50 different pages, you use Blocks.
- Base Template (base.html): Holds the "Skeleton" (Head, Nav, Footer).
- Child Template (dashboard.html): "Extends" the base and fills in the specific "Content Block."
- **Example: `{% extends %}`**

```
{% extends "dashboard.html" %}

{% block content %}
    <h2>Register New Student</h2>

    <form method="POST">
        {% csrf_token %}
        <p>
            <label>Full Name:</label><br>
            <input type="text" name="name" required>
        </p>
        <p>
            <label>Roll Number:</label><br>
            <input type="text" name="roll" required>
        </p>
    </form>
</block>
```

Cross-Site Request Forgery (CSRF)

- ❖ Cross-Site Request Forgery protection is mandatory.
- ❖ Without it, any website could potentially "trick" a logged-in user's browser into sending a POST request to your system.
- ❖ The Tag: {% csrf_token %} inside your HTML.
- ❖ The Middleware: Django automatically checks every POST request for this token. If it's missing or wrong, it returns a 403 Forbidden status code.

```
<h2>Register New Student</h2>

<form method="POST">
  {% csrf_token %}
  <p>
    <label>Full Name:</label><br>
    <input type="text" name="name" required>
  </p>
  <p>
    <label>Roll Number:</label><br>
    <input type="text" name="roll" required>
  </p>
  <p>
```

Session vs Cookies

Feature	Cookie (The Identity Key)	Session (The User Dossier)
Storage Location	Client-side: Stored in the user's browser (Chrome, Safari, etc.).	Server-side: Stored on your Django server (Database or Cache).
Content	Usually just a unique ID string (e.g., <code>sessionid=abc123...</code>).	Full User Data (User ID, permissions, temporary variables).
Security	Lower: Users can see and sometimes edit cookies in browser tools.	Higher: Users cannot see or touch the session data on the server.
Data Limit	Very small (limited to 4KB).	Virtually unlimited (limited only by server storage).
Performance	Fast (sent with every HTTP header).	Slightly slower (requires a database lookup).
Privacy	Can be blocked by users or "incognito" mode.	Controlled entirely by the developer.

Middleware

- ❖ "Custom middleware" refers to user-defined software components integrated into an application's request-response pipeline to handle specific, cross-cutting tasks such as logging, authentication, caching, or modifying requests and responses
- ❖ Middleware as a series of "checkpoints" or "filters" that a request must pass through before it reaches your code (the View), and again before the response goes back to the user.
- ❖ Why?
 - **Authentication:** Checking if a user is logged in before they see any page.
 - **Logging:** Recording which URLs are most popular and who is visiting them.
 - **Performance:** Measuring how many milliseconds a view takes to run to find "slow" pages.
 - **IP Blocking:** Stopping specific malicious users from even reaching your website logic.
 - **Data Formatting:** Automatically converting all incoming request data to lowercase.

Example:

```
import logging
from django.http import JsonResponse

# Set up a standard Python logger
logger = logging.getLogger(__name__)

class StudentSystemMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # --- PHASE 1: LOGGING (Before the view runs) ---
        print(f"Incoming Request: {request.method} to {request.path}")

        # Proceed to the view
        response = self.get_response(request)
        response['X-Student-System-Active'] = 'True'
        # --- PHASE 2: POST-PROCESSING (After the view runs) ---
        return response

    def process_exception(self, request, exception):
        # --- PHASE 3: ERROR HANDLING ---
        # If any view crashes, this function catches it!
        print(f"CRASH DETECTED: {exception}")

        # For our future React integration, we can return JSON instead of a scary error page
        return JsonResponse({
            "error": "Something went wrong on the server.",
            "details": str(exception)
        }, status=500)
```

Example

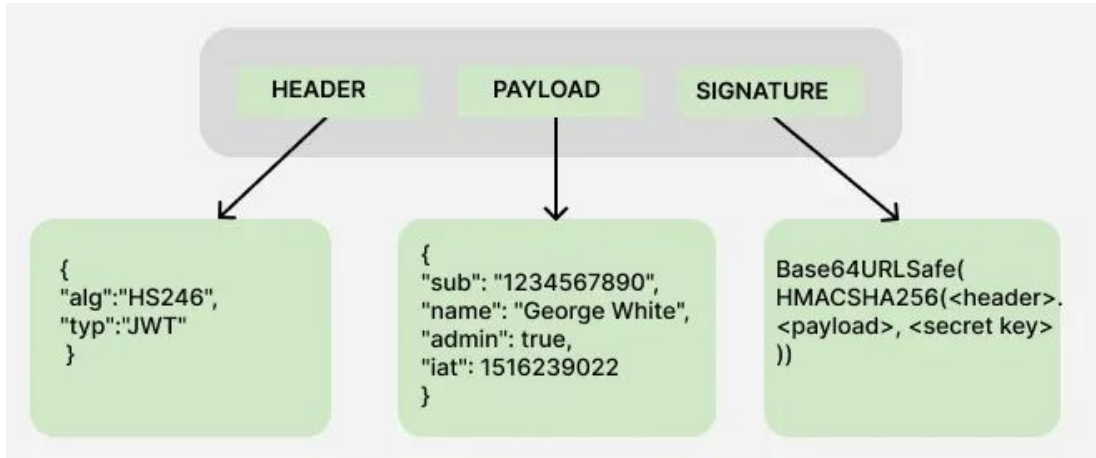
Settings.py

middleware:

```
# custom add  
'studentdashboard.middleware.StudentSystemMiddleware'  
]
```

JSON Web Token (JWT)

- ❖ A JWT is a long, encrypted string that looks like gibberish. It is divided into three parts separated by dots (.):
 - Header: Tells the server which algorithm is used to lock the ticket.
 - Payload: The actual data (e.g., "Student ID: 101", "Name: John").
 - Signature: The "Seal of Authenticity." This proves that the student didn't change their ID from 101 to 999.



JWT: Importance

- ❖ **Mobile & Web:** The same JWT can work for a React web app AND a mobile app at the same time.
- ❖ **Scalability:** If you have 1 million students, your Django server doesn't get slow because it doesn't have to search a "Session Database" every time someone clicks a button.
- ❖ **Cross-Domain:** If your React app is on student-portal.com and your Django API is on api.university.com, JWTs handle this much easier than Cookies.

Framework	Ecosystem	Architecture	Data Handling (ORM)	Concurrency / Speed	Best Industry Use Case
Django	Python	MVT (Model-View-Tem plate)	Django ORM (Powerful/Built-in)	Synchronous (WSGI) / Async support	Content Management, Fintech, Data-Heavy Apps
Flask	Python	Micro-kernel	External (SQLAlchemy)	Synchronous / Lightweight	Microservices, Prototypes, ML Model Deployment
FastAPI	Python	Asynchronous API	External (Pydantic/SQLAlchem y)	High (Asynchronous/ASGI)	High-performance APIs, Real-time Data, AI
Node.js (Express)	JavaScript	Event-Driven	External (Sequelize/Prisma)	Very High (Non-blocking I/O)	Real-time (Chat/Gaming), Streaming, SPAs
Ruby on Rails	Ruby	MVC (Model-View-Cont roller)	ActiveRecord	Multi-threaded	Rapid Prototyping, E-commerce, Startups
.NET MVC	C#	MVC	Entity Framework Core	Very High (Compiled)	Enterprise Software, Banking, Azure Ecosystem
Spring Boot	Java	MVC / Microservices	Spring Data JPA (Hibernate)	Extremely High (Multithreading)	Big Tech (Netflix/Google), Legacy Migration

Heterogeneous System: React js and Django

Backend: **Django**

Install

```
>> pip install django-cors-headers
```